# Pressure Distribution and Force Coefficients of a NACA 0012 Airfoil at Various Angles of Attack

Kaleb Martiny

11/22/2024

MMAE 315: Aerodynamics Lab 9

Dr. Bernhardt

## Abstract

This experiment investigates the aerodynamic performance of the NACA 0012 airfoil by analyzing the pressure distribution along its upper and lower surfaces at varying angles of attack. Lift and drag coefficients were calculated using experimental data and compared against theoretical predictions from thin airfoil theory and published results. Thin airfoil theory, which predicts a linear relationship between lift coefficient and angle of attack for small angles, was used as a benchmark to validate the results. Key trends, such as the increase in lift with angle of attack and the identification of the stall angle at approximately 15°, were successfully observed. However, deviations were noted in the magnitudes of both lift and drag coefficients.

Experimental results showed lift coefficients lower than theoretical and published values, likely due to errors in angle-of-attack alignment, discrepancies in Reynolds number, and possible inaccuracies in data processing. Drag coefficients were higher than expected, suggesting issues with pressure data measurements or incorrect equation implementation. Despite these errors, the experiment captured the core aerodynamic behaviors, including the linearity of lift before stall and the nonlinear drag behavior. Future improvements, such as better control of test conditions and refined computational methods, would reduce these deviations and improve the accuracy of the results to better align with published data.

## Introduction

The NACA 0012 airfoil is one of the most commonly used symmetric airfoils, characterized by its symmetrical profile and zero camber. It has been and is extensively studied to explore the principles of lift and drag, the two primary forces influencing the performance of an aerodynamic body, mainly airfoils. This experiment investigates the airfoil performance by measuring surface pressure distribution at varying angles of attack. These pressure distributions can then be analyzed to compute the lift and drag coefficient of the airfoil. These coefficients are then compared to theoretical predictions based on thin airfoil theory and published experimental results.

Thin airfoil theory is one of the key theories in airfoil analysis. It provides a simple method for predicting the lift characteristics of thin airfoils. The theory models the foil using the camber line and circulation by characterizing the airfoil as thin, within a small angle of attack, and in inviscid flow. Boundary conditions are also applied, primarily the Kutta condition, so that flow around the leading and trailing edges is accurately modeled. One of the results of this theory is the relationship between the lift coefficient and the angle of attack, as given in equation (7). This equation shows that lift is proportional to the angle of attack for a symmetric airfoil. While this theory does

not cover all cases of airfoils and is only useful for small angles of attack, it is valuable for the initial design and analysis of wings, as it gives a quick performance estimate without needing to do more complex calculations (1).

Wind tunnels are essential devices within aerodynamic research. They provide a controlled environment that allows the study of interactions between airflow and objects, such as scale models of airfoils, buildings, and vehicles. They simulate free stream conditions by pushing air through a test section where the object is placed. In the test section, the airflow is constant at every point in the section. In the case of airfoils, wind tunnels allow for precise control of flow speed and angle of attack. This allows for the measurement of forces, pressure distributions, and other flow characteristics (4).

In aerodynamics, lift is generated due to the pressure difference between the airfoil's upper and lower surface, created by variations in the flow velocity around the airfoil. The pressure difference generates a net upward force, which causes the body to lift into the air. Drag is the result of both pressure distributions and surface viscous effects and resists the motion of the airfoil through the air. It is the result of two primary components, pressure and skin friction drag. Pressure drag, like lift, is the result of the pressure distributions around the airfoil, and becomes much more significant when there is flow separation at high angles of attack. Skin friction drag is the result of the interactions between the air and the airfoils surface, caused by the viscous effects of the air. It is these two drag components that make up drag as a whole. For a two-dimensional airfoil, it is also useful to express these forces in terms of non-dimensional coefficients. This allows evaluation across a variety of test conditions. Equations (5) and (6) give the lift and drag coefficients used in this experiment (2).

## Equations

Trapezoidal Lift

(1) $$L = \sum_{lower} \frac{(P-P_\infty)_i+(P-P_\infty)_{i+1}}{2}(c_{i+1}-c_i)cos(\alpha) - \sum_{upper} \frac{(P-P_\infty)_i+(P-P_\infty)_{i+1}}{2}(c_{i+1}-c_i)cos(\alpha)$$

Trapezoidal Drag

(2) $$D = \sum_{lower} \frac{(P-P_\infty)_i+(P-P_\infty)_{i+1}}{2}(c_{i+1}-c_i)sin(\alpha) - \sum_{upper} \frac{(P-P_\infty)_i+(P-P_\infty)_{i+1}}{2}(c_{i+1}-c_i)sin(\alpha)$$

Trapezoidal Lift with Gage Pressure

(3) $$L = \sum_{lower} \frac{P_{gage,i}+P_{gage,i+1}}{2}(c_{i+1}-c_i)cos(\alpha) - \sum_{upper} \frac{P_{gage,i}+P_{gage,i+1}}{2}(c_{i+1}-c_i)cos(\alpha)$$

Trapezoidal Drag with Gage Pressure

(4) $$D = \sum_{lower} \frac{P_{gage,i}+P_{gage,i+1}}{2}(c_{i+1}-c_i)sin(\alpha) - \sum_{upper} \frac{P_{gage,i}+P_{gage,i+1}}{2}(c_{i+1}-c_i)sin(\alpha)$$

Coefficient of Lift

(5) $$C_L = \frac{L}{\frac{1}{2}\rho V^2 c}$$

Coefficient of Drag

(6) $$C_D = \frac{D}{\frac{1}{2}\rho V^2 c}$$

Thin Airfoil Theory

(7) $$C_L = 2\pi\alpha$$

Coefficient of Pressure

(8) $$C_P = \frac{P_{gage}}{q_\infty}$$

Dynamic Pressure

(9) $$q_\infty = \frac{1}{2}\rho V^2$$

## Constants

$L = Lift$

$D = Drag$

$C_L = Coefficient\ of\ Lift$

$C_D = Coefficient\ of\ Drag$

$\alpha = Angle\ of\ Attack\ in\ Degrees$

$P = Surface\ Pressure$

$P_\infty = Static\ Freestream\ Pressure$

$P_{gage} = Difference\ Between\ Surface\ and\ Freestream\ Pressure,\ Recorded\ Pressure$

$c_i = Distance\ to\ the\ pressure\ tap\ from\ the\ leading\ edge$

$c = Chord\ Length$

$\rho = Air\ Density = 1.225\ kg/m^3$

$V = Tunnel\ Velocity = 10\ m/s$

## Experimental Setup

The experimental setup for this lab involves testing a two-dimensional representation of a NACA 0012 airfoil within a controlled, low-speed wind tunnel. The airfoil has several pressure taps along its upper and lower surfaces connected to a Scanivalve DSA 3217 to measure pressure. The use of pressure taps along the surface allows for the recording of the static surface pressure at discrete points along the chord length of the airfoil, which can then be used to calculate the lift and drag coefficients.

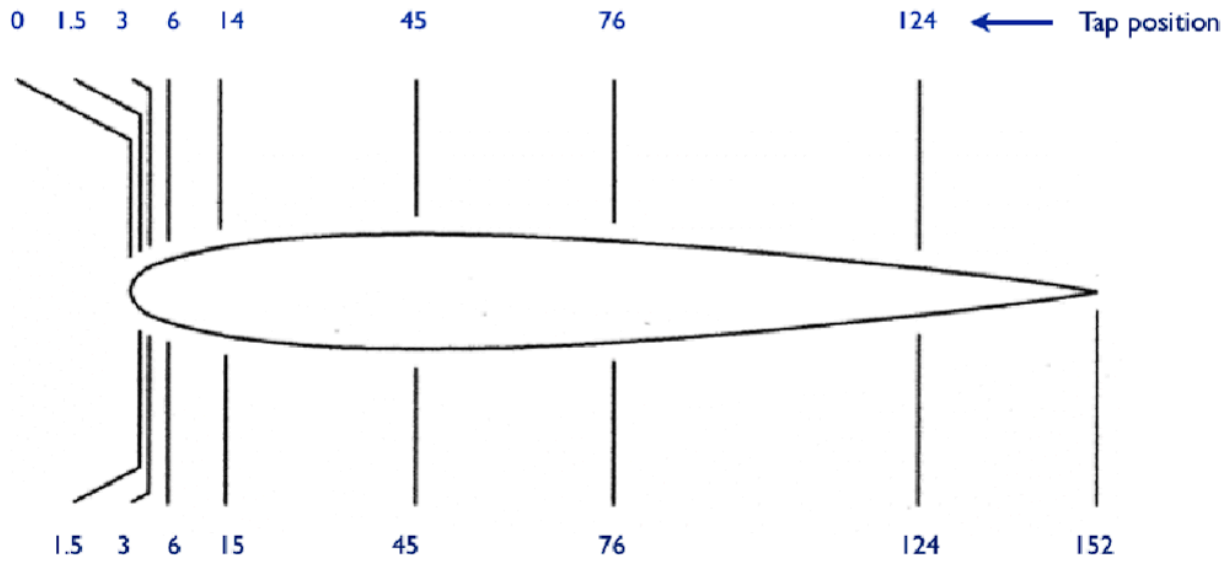



Figure (1) NACA 0012 symmetric airfoil with pressure tap locations. Origin at the leading edge of the airfoil. Chord length at the trailing edge.

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **Top** | 0<br>1 | 1.5 mm<br>9 | 3 mm<br>10 | 6 mm<br>11 | 14 mm<br>12 | 45 mm<br>13 | 76 mm<br>14 | 124 mm<br>15 | 152 mm<br>16 |
| **Bottom** | 0<br>1 | 1.5<br>2 | 3<br>3 | 6<br>4 | 15<br>5 | 45<br>6 | 76<br>7 | 124<br>8 | 152<br>16 |

Figure (2) Measurement of tap locations in millimeters and their assigned number

The wind tunnel generates a free-stream velocity based on an input voltage, with a higher voltage resulting in higher velocity. To correctly set the wind tunnel to the desired velocity of 10 $m/s$, an offset voltage must first be recorded with no flow in the tunnel. This offset voltage will be used in conjunction with equations (8) and (9) to determine the correct input voltage for 10 $m/s$. Once the input voltage has been calculated, the airfoil needs to be set to 0 degrees, and all fittings are secure before the wind tunnel can be started. Once the angle is set and the apparatus has been checked and secured, the controlling potentiometer should be set to its lowest point.

The wind tunnel can then be turned on, and the potentiometer can be turned until the voltage matches the one calculated. This will result in a free stream velocity of 10 $m/s$. Data can now be collected using ScanTel, a computer program that measures the pressures and temperatures read by the taps connected to the Scanivalve. Data should be recorded for a variety of angles of attack. The angles for this experiment are -10, -6, -3, 0, 3, 6, 10, 15, and 20 degrees. It is important to ensure that the set angle of the airfoil is accurate. Using the digital protractor, set the angle, and then let the foil settle in the flow. Readjust the angle if necessary. Once the data has been collected and saved, Python or MATLAB can be used for post-processing. The complete script can be found in Appendix A.

## Results

The goal of doing data analysis on the pressure data gathered from ScanTel is to analyze the performance of the NACA 0012 airfoil through the calculation of the lift and drag coefficients at varying angles of attack. These calculated results can then be compared to theoretical predictions from thin airfoil theory, as well as with published results. The stall angle of the airfoil will also be determined through variations in the lift coefficient. This will give some basic insight into the performance of the airfoil and where its angle of attack limit is.

### Pressure coefficients vs non-dimensional chord length

As shown in Appendix B.1 to B.10, as the angle of attack increases, the difference in pressure between the top and bottom surface generally increases. For the positive angles of attack, this increase is characterized by the upper surface having a large negative coefficient at the leading edge, while the lower surface has a small positive coefficient at the leading edge. For negative angles of attack, this is reversed, with the upper surface having a small positive coefficient at the leading edge, and the lower surface having a large negative coefficient at the leading edge. At no angle of attack, the pressure coefficients on both surfaces are very similar. See Tables 1 and 2 for values used to create all plots in Appendix B.

| | Tap 1 | Tap 9 | Tap 10 | Tap 11 | Tap 12 | Tap 13 | Tap 14 | Tap 15 | Tap 16 |
|---|---|---|---|---|---|---|---|---|---|
| -10 Degrees | -0.80846 | 0.43424 | 0.57561 | 0.23912 | -0.087265 | -0.38772 | -0.4689 | -0.44165 | -0.40723 |
| -6 Degrees | 0.18682 | 0.52099 | 0.39817 | -0.12102 | -0.39366 | -0.54298 | -0.55583 | -0.4757 | -0.38902 |
| -3 Degrees | 0.51201 | 0.3061 | 0.055557 | -0.46518 | -0.64953 | -0.65027 | -0.60374 | -0.4839 | -0.43075 |
| 0 Degrees | 0.53364 | -0.25982 | -0.48821 | -0.91236 | -0.96521 | -0.79403 | -0.69997 | -0.58059 | -0.49901 |
| 3 Degrees | 0.21922 | -1.1868 | -1.2716 | -1.4348 | -1.3059 | -0.92117 | -0.80793 | -0.59707 | -0.37732 |
| 6 Degrees | -0.40397 | -2.2783 | -2.0945 | -1.8914 | -1.5782 | -0.97492 | -0.7909 | -0.59407 | -0.3523 |
| 10 Degrees | -1.4513 | -3.3825 | -2.6297 | -2.4835 | -2.5387 | -1.1266 | -0.83595 | -0.61295 | -0.45461 |
| 15 Degrees | -0.70752 | -1.5944 | -1.2998 | -1.2422 | -1.2864 | -1.1742 | -1.2357 | -1.1806 | -0.98869 |
| 20 Degrees | -0.80275 | -1.1645 | -1.0455 | -1.0375 | -1.0856 | -1.0851 | -1.1363 | -1.1216 | -1.1181 |

Table (1) Coefficients of pressure across all angles of attack along the upper surface of the NACA 0012 airfoil. Tap 1 sits on the leading edge. Tap 16 sits on the trailing edge.

| | Tap 1 | Tap 2 | Tap 3 | Tap 4 | Tap 5 | Tap 6 | Tap 7 | Tap 8 | Tap 16 |
|---|---|---|---|---|---|---|---|---|---|
| -10 Degrees | -0.80846 | -2.9424 | -2.823 | -2.4103 | -2.1598 | -1.1681 | -0.91395 | -0.66466 | -0.40723 |
| -6 Degrees | 0.18682 | -1.249 | -1.5646 | -1.8419 | -1.529 | -1.0718 | -0.92986 | -0.62282 | -0.38902 |
| -3 Degrees | 0.51201 | -0.28696 | -0.75125 | -1.3256 | -1.1916 | -0.92911 | -0.77905 | -0.66066 | -0.43075 |
| 0 Degrees | 0.53364 | 0.35698 | -0.081988 | -0.82666 | -0.84325 | -0.76257 | -0.66217 | -0.53314 | -0.49901 |
| 3 Degrees | 0.21922 | 0.58822 | 0.38528 | -0.39121 | -0.51775 | -0.60416 | -0.57038 | -0.47031 | -0.37732 |
| 6 Degrees | -0.40397 | 0.52721 | 0.59304 | -0.075419 | -0.26026 | -0.46919 | -0.49264 | -0.44031 | -0.3523 |
| 10 Degrees | -1.4513 | 0.043468 | 0.62245 | 0.22181 | 0.020288 | -0.30129 | -0.39746 | -0.41404 | -0.45461 |
| 15 Degrees | -0.70752 | 0.30423 | 0.6192 | 0.18982 | 0.0045393 | -0.3385 | -0.48291 | -0.57738 | -0.98869 |
| 20 Degrees | -0.80275 | 0.12869 | 0.59647 | 0.2914 | 0.12904 | -0.26198 | -0.46005 | -0.61564 | -1.1181 |

Table (2) Coefficients of pressure across all angles of attack along the lower surface of the NACA 0012 airfoil. Tap 1 sits on the leading edge. Tap 16 sits on the trailing edge. The results tabulated above were calculated using equations (8) and (9), with the gage pressure being taken directly from the imported data from ScanTel.

## Lift coefficient vs angle of attack

As shown in Appendix B.12 and B.13, the calculated lift coefficient has a linear relationship with the angle of attack up until $C_L$ max at 10 degrees, after which the lift coefficient drops off significantly. Equations (3) and (5) were used to calculate the lift coefficients at all angles of attack. The upper and lower surfaces were computed independently from each other to simplify the calculation and then subtracted from each other per equation (3). The calculation for all lift coefficients can be found in Appendix A.1.

| | -10 | -6 | -3 | 0 | 3 | 6 | 10 | 15 | 20 |
|---|---|---|---|---|---|---|---|---|---|
| Lift Coefficient | -0.76818 | -0.49607 | -0.26602 | 0.058742 | 0.33923 | 0.51978 | 0.82905 | 0.75648 | 0.6699 |

Table (3) Coefficients of lift at all angles of attack for the NACA 0012 airfoil in subsonic flow.

## Drag coefficient vs lift coefficient

In conjunction with the previously calculated lift coefficient, the drag coefficient was also calculated using equations (4) and (6). They were then plotted against each other for use in later analysis. See Appendix B.11 for the plotted relationship. Table (4) is the result of equations (4) and (6)

| | -10 | -6 | -3 | 0 | 3 | 6 | 10 | 15 | 20 |
|---|---|---|---|---|---|---|---|---|---|
| Drag Coefficient | 0.13545 | 0.052139 | 0.013942 | 0 | 0.017778 | 0.054631 | 0.14618 | 0.2027 | 0.24383 |

Table (4) Coefficients of drag at all angles of attack for the NACA 0012 airfoil in subsonic flow.

## Stall angle

Based on the result shown in Appendix B.12, the stall angle was found to be about 15 degrees. This angle was chosen for three reasons. The first is that it is the first point to significantly drop in value after the maximum at 10 degrees. The second is that in Appendix B.11, it is the first lift coefficient to regress. The third reason is that through panel analysis in xflr5, 15 degrees is the first angle to see more significant flow separation on the trailing edge.

## Discussion of Results

Looking at the pressure coefficients vs non-dimensional chord length graphs in Appendix B.1 to B.10, it is clear that the results gathered generally match what the theory predicts. Looking at the upper surface of the airfoil for positive angles of attack, the lift coefficient is significantly more negative than the lower surface is positive on the leading edge. This is characteristic behavior of a lifting airfoil and matches results from sources like xflr5. This pressure distribution causes a negative pressure on the top surface and a small positive pressure on the lower surface. It is this pressure difference that causes lift in the upward direction. Looking at the upper surface for negative angles of attack, the lift coefficient is slightly positive, while the lower surface is significantly more negative than the upper surface is positive on the leading edge. For a negative angle of attack, this is characteristic and matches analysis by xflr5. This pressure distribution causes lift in the downward direction. At zero angle of attack, both the upper and lower surfaces are nearly identical. This is expected because the NACA 0012 is a symmetric airfoil, so having zero angle of attack should result in equal pressure as the fluid moves around the upper and lower surfaces at the same speed.

For the lift coefficient vs angle of attack in Appendix B.12 and B.13, the resulting line is what is expected. The airfoil exhibits a linear increase in the lift coefficient up until the critical angle, after which the lift coefficient drops off rapidly. This matches published results as seen in Appendix D.2, as well as follows thin airfoil theory, as described by equation (7). Appendix B.13 plots the calculated data, thin airfoil theory, and published results for clear comparison.

For the coefficient of drag vs coefficient of lift in Appendix B.11, the resulting shape is generally what is expected. While thin airfoil theory does not offer an estimation of the drag coefficient, it compares well to the published results as shown in Appendix D.2. This calculated plot was used to determine the stall angle of the airfoil by taking the first point after $C_{L,max}$, or the maximum lift coefficient.

While the results obtained generally match the expected results, there are significant errors. The first is in the coefficient of lift vs angle of attack. As shown in Appendix B.13, there is a significant deviation between the calculated results, and what is predicted by thin airfoil theory and published by Abbott and von Doenhoff. While a specific source of this error has not been identified, some guesses can be made as to where the deviations have come from. One source of potential error is simply in the calculation of the coefficient. While the code was checked several times for accuracy, there is a chance that there is something slightly wrong. The code assumes that the lift and drag equations are calculated just that, lift and drag, but if they calculate lift and drag per unit span, then equations (5) and (6) might not be the correct equations. Another source of potential error is that the angles for the data might not be correct, or that the airfoil was not properly set to the correct angle. As the lift and drag calculations use the angle of attack directly, deviations in the value of the angle of attack from the expected would cause calculation errors. A third source of potential error could stem from the setup conditions. If the Reynolds number that the experimental results were calculated at differs significantly from the Reynolds number of published results, then the values calculated will differ significantly.

Another significant error in this experiment lies in the coefficient of drag. As mentioned previously, the code was checked several times for accuracy, however there may have been calculation errors. Looking at Appendix B.11 and Table (4), the values of the drag coefficient are substantially larger than those found in Appendix D.1, which contains published results. This error shares similar reasoning to that of the lift coefficient, however, it must be said that the error between the two cannot solely be based on improper data collection alone. If that were the case, there would be significant errors in both coefficients, not just one.

A third notable source of error involves the pressure coefficients for the various angles of attack. As previously mentioned, in general, the data gathered looks correct,

however when compared to xflr5, the magnitude of the calculated coefficients does not match that of computational analysis, and is smaller than expected. Again, this could be the result of Reynolds number differences, but more likely than not the error stems from the calculations or data collection.

The stall angle of the airfoil matched that predicted by xflr5 and what the consensus is for the stall angle of the NACA 0012 airfoil. No significant source of error was seen here.

## Conclusion

The results of this experiment provided valuable insights into the aerodynamic performance of the NACA 0012 airfoil under varying angles of attack. The calculated lift coefficients showed a linear relationship with the angle of attack up to the critical angle of approximately 10 degrees, after which the lift coefficient dropped significantly, an indicator of stall. This behavior aligns with expectations based on thin airfoil theory and published experimental results, though some deviations were observed. The calculated coefficients of lift were generally lower than both theoretical predictions and published data, which may result from potential errors in angle-of-attack alignment, discrepancies in Reynolds number, or incorrect implementation of data processing and analysis. Additionally, the drag coefficients were notably higher than those in published results, suggesting possible inaccuracies in pressure measurements, computational methods, or equation setup. Despite these errors, the experiment successfully captured the key trends in lift and drag behavior, including the identification of the stall angle at approximately 15°, which matched theoretical predictions and computational analysis. Future improvements to the experimental setup and data processing methods, such as ensuring precise angle-of-attack alignment and matching Reynolds number conditions, would increase the accuracy of results and allow for closer agreement with theoretical and published data.

## References

(1) Anderson, J. D., & Cadou, C. (2024). *Fundamentals of aerodynamics*. McGraw-Hill.
(2) Gerhart, P. M. (2016). *Fundamentals of Fluid Mechanics Munson*. Wiley.
(3) NASA. (n.d.-a). *2D NACA 0012 airfoil validation*. NASA.

https://turbmodels.larc.nasa.gov/naca0012_val.html

(4) NASA. (n.d.-b). *Wind tunnel*. NASA.

https://www.grc.nasa.gov/www/k-12/airplane/tunnel1.html

# Appendix A - MATLAB Script

```
% Pressure Coefficients

% Loads the data
offset_data = readtable('offset.CSV');
alpha_neg10_data = readtable('aoaneg10.CSV');
alpha_neg6_data = readtable('aoaneg6.CSV');
alpha_neg3_data = readtable('aoaneg3.CSV');
alpha_0_data = readtable('aoa0.CSV');
alpha_3_data = readtable('aoa3.CSV');
alpha_6_data = readtable('aoa6.CSV');
alpha_10_data = readtable('aoa10.CSV');
alpha_15_data = readtable('aoa15.CSV');
alpha_20_data = readtable('aoa20.CSV');

% Pulls pressures from data and takes the mean of each tap, converted to Pa
offset = mean(offset_data(:, 2:17)) .* 6894.76;
    offset_upper = table2array(offset(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14"
"P15" "P16"]));
    offset_lower = table2array(offset(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));

alpha_neg10 = mean(alpha_neg10_data(:, 2:17)) .* 6894.76;
    alpha_neg10_upper = table2array(alpha_neg10(1, ["P1" "P9" "P10" "P11" "P12"
"P13" "P14" "P15" "P16"]));
    alpha_neg10_lower = table2array(alpha_neg10(1, ["P1" "P2" "P3" "P4" "P5" "P6"
"P7" "P8" "P16"]));

alpha_neg6 = mean(alpha_neg6_data(:, 2:17)) .* 6894.76;
    alpha_neg6_upper = table2array(alpha_neg6(1, ["P1" "P9" "P10" "P11" "P12" "P13"
"P14" "P15" "P16"]));
    alpha_neg6_lower = table2array(alpha_neg6(1, ["P1" "P2" "P3" "P4" "P5" "P6"
"P7" "P8" "P16"]));

alpha_neg3 = mean(alpha_neg3_data(:, 2:17)) .* 6894.76;
    alpha_neg3_upper = table2array(alpha_neg3(1, ["P1" "P9" "P10" "P11" "P12" "P13"
"P14" "P15" "P16"]));
    alpha_neg3_lower = table2array(alpha_neg3(1, ["P1" "P2" "P3" "P4" "P5" "P6"
"P7" "P8" "P16"]));

alpha_0 = mean(alpha_0_data(:, 2:17)) .* 6894.76;
    alpha_0_upper = table2array(alpha_0(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14"
"P15" "P16"]));
    alpha_0_lower = table2array(alpha_0(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));

alpha_3 = mean(alpha_3_data(:, 2:17)) .* 6894.76;
    alpha_3_upper = table2array(alpha_3(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14"
"P15" "P16"]));
    alpha_3_lower = table2array(alpha_3(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));
```

```
alpha_6 = mean(alpha_6_data(:, 2:17)) .* 6894.76;
    alpha_6_upper = table2array(alpha_6(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14"
"P15" "P16"]));
    alpha_6_lower = table2array(alpha_6(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));

alpha_10 = mean(alpha_10_data(:, 2:17)) .* 6894.76;
    alpha_10_upper = table2array(alpha_10(1, ["P1" "P9" "P10" "P11" "P12" "P13"
"P14" "P15" "P16"]));
    alpha_10_lower = table2array(alpha_10(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7"
"P8" "P16"]));

alpha_15 = mean(alpha_15_data(:, 2:17)) .* 6894.76;
    alpha_15_upper = table2array(alpha_15(1, ["P1" "P9" "P10" "P11" "P12" "P13"
"P14" "P15" "P16"]));
    alpha_15_lower = table2array(alpha_15(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7"
"P8" "P16"]));

alpha_20 = mean(alpha_20_data(:, 2:17)) .* 6894.76;
    alpha_20_upper = table2array(alpha_20(1, ["P1" "P9" "P10" "P11" "P12" "P13"
"P14" "P15" "P16"]));
    alpha_20_lower = table2array(alpha_20(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7"
"P8" "P16"]));

% Clears the raw data from memory for clarity
clear offset_data alpha_neg10_data alpha_neg6_data alpha_neg3_data alpha_0_data ...
    alpha_3_data alpha_6_data alpha_10_data alpha_15_data alpha_20_data
```

```
% Knowns
wind_tunnel_velocity = 10; % m/s
density = 1.225; % kg/m^3
tap_locations_upper = [0 1.5 3 6 14 45 76 124 152] ./ 1000; % m
tap_locations_lower = [0 1.5 3 6 15 45 76 124 152] ./ 1000; % m
c = 152 / 1000; % m

% Angles of attack
alpha_deg = [-10 -6 -3 0 3 6 10 15 20];
alpha = deg2rad(alpha_deg);

q_inf = 0.5 * density * wind_tunnel_velocity^2;

cp_neg10 = alpha_neg10 ./ q_inf;
    cp_neg10_upper = table2array(cp_neg10(1, ["P1" "P9" "P10" "P11" "P12" "P13"
"P14" "P15" "P16"]));
    cp_neg10_lower = table2array(cp_neg10(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7"
"P8" "P16"]));

cp_neg6 = alpha_neg6 ./ q_inf;
```

```
    cp_neg6_upper = table2array(cp_neg6(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14"
"P15" "P16"]));
    cp_neg6_lower = table2array(cp_neg6(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));

cp_neg3 = alpha_neg3 ./ q_inf;
    cp_neg3_upper = table2array(cp_neg3(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14"
"P15" "P16"]));
    cp_neg3_lower = table2array(cp_neg3(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));

cp_0 = alpha_0 ./ q_inf;
    cp_0_upper = table2array(cp_0(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14" "P15"
"P16"]));
    cp_0_lower = table2array(cp_0(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));

cp_3 = alpha_3 ./ q_inf;
    cp_3_upper = table2array(cp_3(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14" "P15"
"P16"]));
    cp_3_lower = table2array(cp_3(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));

cp_6 = alpha_6 ./ q_inf;
    cp_6_upper = table2array(cp_6(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14" "P15"
"P16"]));
    cp_6_lower = table2array(cp_6(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));

cp_10 = alpha_10 ./ q_inf;
    cp_10_upper = table2array(cp_10(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14"
"P15" "P16"]));
    cp_10_lower = table2array(cp_10(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));

cp_15 = alpha_15 ./ q_inf;
    cp_15_upper = table2array(cp_15(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14"
"P15" "P16"]));
    cp_15_lower = table2array(cp_15(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));

cp_20 = alpha_20./ q_inf;
    cp_20_upper = table2array(cp_20(1, ["P1" "P9" "P10" "P11" "P12" "P13" "P14"
"P15" "P16"]));
    cp_20_lower = table2array(cp_20(1, ["P1" "P2" "P3" "P4" "P5" "P6" "P7" "P8"
"P16"]));


x_c_upper = tap_locations_upper / 152;
x_c_lower = tap_locations_lower / 152;
```

```
figure(1)
plot(x_c_upper, cp_neg10_upper)
hold on
plot(x_c_lower, cp_neg10_lower)
title('Coefficient of Pressure vs x/C at -10°')
set(gca, 'Ydir', 'reverse')
legend('Upper', 'Lower', 'Location', 'Southeast')
xlabel('x/C')
ylabel('Cp')
hold off

figure(2)
plot(x_c_upper, cp_neg6_upper)
hold on
plot(x_c_lower, cp_neg6_lower)
title('Coefficient of Pressure vs x/C at -6°')
set(gca, 'Ydir', 'reverse')
legend('Upper', 'Lower', 'Location', 'Southeast')
xlabel('x/C')
ylabel('Cp')
hold off

figure(3)
plot(x_c_upper, cp_neg3_upper)
hold on
plot(x_c_lower, cp_neg3_lower)
title('Coefficient of Pressure vs x/C at -3°')
set(gca, 'Ydir', 'reverse')
legend('Upper', 'Lower', 'Location', 'Southeast')
xlabel('x/C')
ylabel('Cp')
hold off

figure(4)
plot(x_c_upper, cp_0_upper)
hold on
plot(x_c_lower, cp_0_lower)
title('Coefficient of Pressure vs x/C at 0°')
set(gca, 'Ydir', 'reverse')
legend('Upper', 'Lower', 'Location', 'Southeast')
xlabel('x/C')
ylabel('Cp')
hold off

figure(5)
plot(x_c_upper, cp_3_upper)
hold on
```

```
plot(x_c_lower, cp_3_lower)
title('Coefficient of Pressure vs x/C at 3°')
set(gca, 'Ydir', 'reverse')
legend('Upper', 'Lower', 'Location', 'Southeast')
xlabel('x/C')
ylabel('Cp')
hold off

figure(6)
plot(x_c_upper, cp_6_upper)
hold on
plot(x_c_lower, cp_6_lower)
title('Coefficient of Pressure vs x/C at 6°')
set(gca, 'Ydir', 'reverse')
legend('Upper', 'Lower', 'Location', 'Southeast')
xlabel('x/C')
ylabel('Cp')
hold off

figure(7)
plot(x_c_upper, cp_10_upper)
hold on
plot(x_c_lower, cp_10_lower)
title('Coefficient of Pressure vs x/C at 10°')
set(gca, 'Ydir', 'reverse')
legend('Upper', 'Lower', 'Location', 'Southeast')
xlabel('x/C')
ylabel('Cp')
hold off

figure(8)
plot(x_c_upper, cp_15_upper)
hold on
plot(x_c_lower, cp_15_lower)
title('Coefficient of Pressure vs x/C at 15°')
set(gca, 'Ydir', 'reverse')
legend('Upper', 'Lower', 'Location', 'Southeast')
xlabel('x/C')
ylabel('Cp')
hold off

figure(9)
plot(x_c_upper, cp_20_upper)
hold on
plot(x_c_lower, cp_20_lower)
title('Coefficient of Pressure vs x/C at 20°')
set(gca, 'Ydir', 'reverse')
legend('Upper', 'Lower', 'Location', 'Southeast')
xlabel('x/C')
ylabel('Cp')
```

```
hold off


figure(10)
plot(x_c_upper, cp_neg10_upper)
hold on
plot(x_c_lower, cp_neg10_lower)
plot(x_c_upper, cp_neg6_upper)
plot(x_c_lower, cp_neg6_lower)
plot(x_c_upper, cp_neg3_upper)
plot(x_c_lower, cp_neg3_lower)
plot(x_c_upper, cp_0_upper)
plot(x_c_lower, cp_0_lower)
plot(x_c_upper, cp_3_upper)
plot(x_c_lower, cp_3_lower)
plot(x_c_upper, cp_6_upper)
plot(x_c_lower, cp_6_lower)
plot(x_c_upper, cp_10_upper)
plot(x_c_lower, cp_10_lower)
plot(x_c_upper, cp_15_upper)
plot(x_c_lower, cp_15_lower)
plot(x_c_upper, cp_20_upper)
plot(x_c_lower, cp_20_lower)
title('Coefficient of Pressure vs x/C all angles')
xlabel('x/C')
ylabel('Cp')
set(gca, 'Ydir', 'reverse')
legend('-10°','-6°','-3°','0°','3°','6°','10°','15°','20°', 'Location', 'northeast')

pressure_table_upper = zeros(9,9);
pressure_table_lower = zeros(9,9);

pressure_table_upper(1,:) = cp_neg10_upper;
pressure_table_upper(2,:) = cp_neg6_upper;
pressure_table_upper(3,:) = cp_neg3_upper;
pressure_table_upper(4,:) = cp_0_upper;
pressure_table_upper(5,:) = cp_3_upper;
pressure_table_upper(6,:) = cp_6_upper;
pressure_table_upper(7,:) = cp_10_upper;
pressure_table_upper(8,:) = cp_15_upper;
pressure_table_upper(9,:) = cp_20_upper;

pressure_table_lower(1,:) = cp_neg10_lower;
pressure_table_lower(2,:) = cp_neg6_lower;
pressure_table_lower(3,:) = cp_neg3_lower;
pressure_table_lower(4,:) = cp_0_lower;
pressure_table_lower(5,:) = cp_3_lower;
pressure_table_lower(6,:) = cp_6_lower;
pressure_table_lower(7,:) = cp_10_lower;
pressure_table_lower(8,:) = cp_15_lower;
```

```
pressure_table_lower(9,:) = cp_20_lower;


pressure_table_upper = array2table(pressure_table_upper);
pressure_table_lower = array2table(pressure_table_lower);

pressure_table_upper.Properties.VariableNames = {'Tap 1', 'Tap 9', 'Tap 10', 'Tap
11', 'Tap 12', 'Tap 13', 'Tap 14', 'Tap 15', 'Tap 16'};
pressure_table_upper.Properties.RowNames = {'-10 Degrees', '-6 Degrees', '-3
Degrees', '0 Degrees', '3 Degrees', '6 Degrees', '10 Degrees', '15 Degrees', '20
Degrees'};

pressure_table_lower.Properties.VariableNames = {'Tap 1', 'Tap 2', 'Tap 3', 'Tap
4', 'Tap 5', 'Tap 6', 'Tap 7', 'Tap 8', 'Tap 16'};
pressure_table_lower.Properties.RowNames = {'-10 Degrees', '-6 Degrees', '-3
Degrees', '0 Degrees', '3 Degrees', '6 Degrees', '10 Degrees', '15 Degrees', '20
Degrees'};

disp(pressure_table_upper)
disp(pressure_table_lower)

% Lift and Drag Coefficients

lift_lower = zeros(1, length(alpha_0_lower)-1);
lift_upper = zeros(1, length(alpha_0_upper)-1);
drag_lower = zeros(1, length(alpha_0_lower)-1);
drag_upper = zeros(1, length(alpha_0_lower)-1);

cl = zeros(1, 9);
cd = zeros(1, 9);

for i = 1 : length(offset_lower)-1

    lift_lower(i) = (((alpha_neg10_lower(i) + alpha_neg10_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * cos(alpha(1)));
    lift_upper(i) = (((alpha_neg10_upper(i) + alpha_neg10_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * cos(alpha(1)));

    drag_lower(i) = (((alpha_neg10_lower(i) + alpha_neg10_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * sin(alpha(1)));
    drag_upper(i) = (((alpha_neg10_upper(i) + alpha_neg10_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * sin(alpha(1)));

    lift_low = sum(lift_lower);
    lift_up = sum(lift_upper);
    drag_low = sum(drag_lower);
    drag_up = sum(drag_upper);

    lift = lift_low - lift_up;
    drag = drag_low + drag_up;
```

```
    cl(1) = lift / (q_inf * c);
    cd(1) = drag / (q_inf * c);

end

for i = 1 : length(offset_lower)-1

    lift_lower(i) = (((alpha_neg6_lower(i) + alpha_neg6_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * cos(alpha(2)));
    lift_upper(i) = (((alpha_neg6_upper(i) + alpha_neg6_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * cos(alpha(2)));

    drag_lower(i) = (((alpha_neg6_lower(i) + alpha_neg6_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * sin(alpha(2)));
    drag_upper(i) = (((alpha_neg6_upper(i) + alpha_neg6_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * sin(alpha(2)));

    lift_low = sum(lift_lower);
    lift_up = sum(lift_upper);
    drag_low = sum(drag_lower);
    drag_up = sum(drag_upper);

    lift = lift_low - lift_up;
    drag = drag_low - drag_up;

    cl(2) = lift / (q_inf * c);
    cd(2) = drag / (q_inf * c);

end

for i = 1 : length(offset_lower)-1

    lift_lower(i) = (((alpha_neg3_lower(i) + alpha_neg3_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * cos(alpha(3)));
    lift_upper(i) = (((alpha_neg3_upper(i) + alpha_neg3_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * cos(alpha(3)));

    drag_lower(i) = (((alpha_neg3_lower(i) + alpha_neg3_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * sin(alpha(3)));
    drag_upper(i) = (((alpha_neg3_upper(i) + alpha_neg3_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * sin(alpha(3)));

    lift_low = sum(lift_lower);
    lift_up = sum(lift_upper);
    drag_low = sum(drag_lower);
    drag_up = sum(drag_upper);

    lift = lift_low - lift_up;
    drag = drag_low - drag_up;
```

```
    cl(3) = lift / (q_inf * c);
    cd(3) = drag / (q_inf * c);

end

for i = 1 : length(offset_lower)-1

    lift_lower(i) = (((alpha_0_lower(i) + alpha_0_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * cos(alpha(4)));
    lift_upper(i) = (((alpha_0_upper(i) + alpha_0_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * cos(alpha(4)));

    drag_lower(i) = (((alpha_0_lower(i) + alpha_0_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * sin(alpha(4)));
    drag_upper(i) = (((alpha_0_upper(i) + alpha_0_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * sin(alpha(4)));

    lift_low = sum(lift_lower);
    lift_up = sum(lift_upper);
    drag_low = sum(drag_lower);
    drag_up = sum(drag_upper);

    lift = lift_low - lift_up;
    drag = drag_low - drag_up;

    cl(4) = lift / (q_inf * c);
    cd(4) = drag / (q_inf * c);

end

for i = 1 : length(offset_lower)-1

    lift_lower(i) = (((alpha_3_lower(i) + alpha_3_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * cos(alpha(5)));
    lift_upper(i) = (((alpha_3_upper(i) + alpha_3_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * cos(alpha(5)));

    drag_lower(i) = (((alpha_3_lower(i) + alpha_3_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * sin(alpha(5)));
    drag_upper(i) = (((alpha_3_upper(i) + alpha_3_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * sin(alpha(5)));

    lift_low = sum(lift_lower);
    lift_up = sum(lift_upper);
    drag_low = sum(drag_lower);
    drag_up = sum(drag_upper);

    lift = lift_low - lift_up;
    drag = drag_low - drag_up;
```

```
    cl(5) = lift / (q_inf * c);
    cd(5) = drag / (q_inf * c);

end

for i = 1 : length(offset_lower)-1

    lift_lower(i) = (((alpha_6_lower(i) + alpha_6_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * cos(alpha(6)));
    lift_upper(i) = (((alpha_6_upper(i) + alpha_6_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * cos(alpha(6)));

    drag_lower(i) = (((alpha_6_lower(i) + alpha_6_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * sin(alpha(6)));
    drag_upper(i) = (((alpha_6_upper(i) + alpha_6_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * sin(alpha(6)));

    lift_low = sum(lift_lower);
    lift_up = sum(lift_upper);
    drag_low = sum(drag_lower);
    drag_up = sum(drag_upper);

    lift = lift_low - lift_up;
    drag = drag_low - drag_up;

    cl(6) = lift / (q_inf * c);
    cd(6) = drag / (q_inf * c);

end

for i = 1 : length(offset_lower)-1

    lift_lower(i) = (((alpha_10_lower(i) + alpha_10_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * cos(alpha(7)));
    lift_upper(i) = (((alpha_10_upper(i) + alpha_10_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * cos(alpha(7)));

    drag_lower(i) = (((alpha_10_lower(i) + alpha_10_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * sin(alpha(7)));
    drag_upper(i) = (((alpha_10_upper(i) + alpha_10_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * sin(alpha(7)));

    lift_low = sum(lift_lower);
    lift_up = sum(lift_upper);
    drag_low = sum(drag_lower);
    drag_up = sum(drag_upper);

    lift = lift_low - lift_up;
    drag = drag_low - drag_up;
```

```
    cl(7) = lift / (q_inf * c);
    cd(7) = drag / (q_inf * c);

end

for i = 1 : length(offset_lower)-1

    lift_lower(i) = (((alpha_15_lower(i) + alpha_15_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * cos(alpha(8)));
    lift_upper(i) = (((alpha_15_upper(i) + alpha_15_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * cos(alpha(8)));

    drag_lower(i) = (((alpha_15_lower(i) + alpha_15_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * sin(alpha(8)));
    drag_upper(i) = (((alpha_15_upper(i) + alpha_15_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * sin(alpha(8)));

    lift_low = sum(lift_lower);
    lift_up = sum(lift_upper);
    drag_low = sum(drag_lower);
    drag_up = sum(drag_upper);

    lift = lift_low - lift_up;
    drag = drag_low - drag_up;

    cl(8) = lift / (q_inf * c);
    cd(8) = drag / (q_inf * c);

end

for i = 1 : length(offset_lower)-1

    lift_lower(i) = (((alpha_20_lower(i) + alpha_20_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * cos(alpha(9)));
    lift_upper(i) = (((alpha_20_upper(i) + alpha_20_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * cos(alpha(9)));

    drag_lower(i) = (((alpha_20_lower(i) + alpha_20_lower(i+1)) / 2) *
(tap_locations_lower(i+1) - tap_locations_lower(i)) * sin(alpha(9)));
    drag_upper(i) = (((alpha_20_upper(i) + alpha_20_upper(i+1)) / 2) *
(tap_locations_upper(i+1) - tap_locations_upper(i)) * sin(alpha(9)));

    lift_low = sum(lift_lower);
    lift_up = sum(lift_upper);
    drag_low = sum(drag_lower);
    drag_up = sum(drag_upper);

    lift = lift_low - lift_up;
    drag = drag_low - drag_up;
```

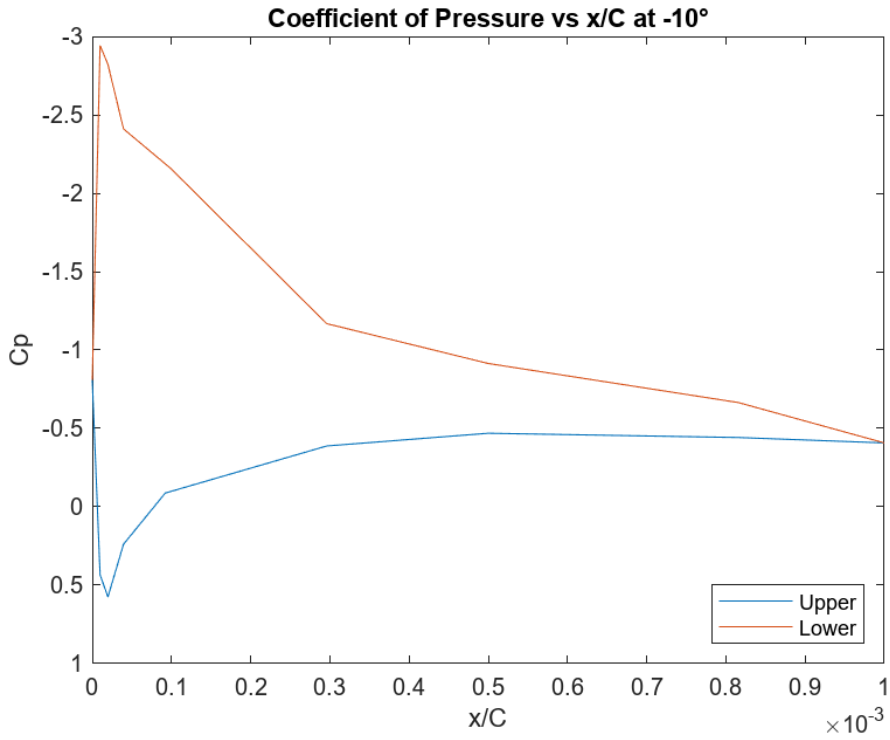
```
    cl(9) = lift / (q_inf * c);
    cd(9) = drag / (q_inf * c);

end

cl_table = array2table(cl);
cd_table = array2table(cd);


cl_table.Properties.VariableNames = {'-10', '-6', '-3', '0', '3', '6', '10', '15',
'20'};
cl_table.Properties.RowNames = {'Lift Coefficient'};

cd_table.Properties.VariableNames = {'-10', '-6', '-3', '0', '3', '6', '10', '15',
'20'};
cd_table.Properties.RowNames = {'Drag Coefficient'};

disp(cd_table)
disp(cl_table)


cl_cd= [-1.25583 0.0127127
-1.09305 0.00986522
-0.860237 0.00856238
-0.653233 0.00751899
-0.549778 0.00691128
-0.446183 0.00656162
-0.342588 0.00621197
-0.230298 0.00594787
-0.100665 0.00576885
0.0205051 0.00593438
0.115637 0.00592927
0.262707 0.00600738
0.461945 0.00659882
0.557543 0.0074539
0.661509 0.00779239
0.878463 0.00915707
1.07854 0.0112969
1.26127 0.0133515];

von_cl = cl_cd(:,1);
von_cd = cl_cd(:,2);


thin = 2*pi*alpha(1:7);

figure(12)
plot(von_cl, von_cd, 'Color', 'Blue')
hold on
```

```
plot(von_cl, von_cd, '.k', 'MarkerSize', 12)
title('Coefficient of Drag vs Coefficient of Lift von Doenhoff')
xlabel('Coefficient of Lift')
ylabel('Coefficient of Drag')
grid on
hold off

figure(13)
plot(cl, cd, 'color', 'Blue')
hold on
plot(cl, cd, '.k', 'MarkerSize', 12)
title('Coefficient of Drag vs Coefficient of Lift')
xlabel('Coefficient of Lift')
ylabel('Coefficient of Drag')
grid on
hold off

stall_point = cl(8);
stall_angle = alpha_deg(8);

figure(14)
plot(alpha_deg, cl, 'Color', 'Blue')
hold on
plot(alpha_deg, cl, '.k', 'MarkerSize', 12)
plot(stall_angle, stall_point, 'r.', 'MarkerSize', 17)
title('Coefficient of lift vs Angle of attack with stall point')
xlabel('Angle of attack (Deg)')
ylabel('Coefficient of Lift')
grid on
legend('','','Stall Angle', 'Location', 'Southeast')
hold off

von_doenhoff =[-17.2794 -1.25323
-16.2296 -1.34704
-15.8616 -1.54416
-15.1713 -1.51805
-14.3133 -1.44038
-13.2811 -1.3712
-12.2535 -1.25912
-11.2222 -1.18135
-10.1947 -1.06927
-8.14138 -0.827958
-6.25579 -0.638207
-5.22822 -0.526128
-4.19972 -0.422627
-1.96944 -0.215533
0.       0.
0.940006 0.120611
1.96944 0.215533
2.99515 0.34477
```

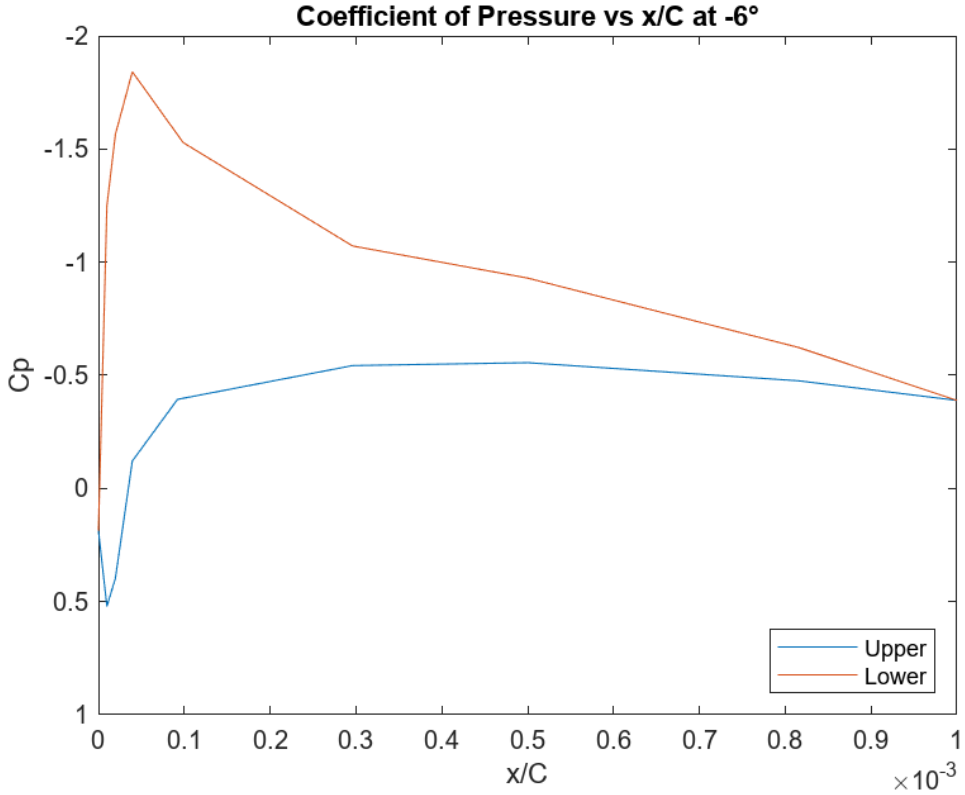
```
3.85131 0.439599
4.87888 0.551678
5.90831 0.6466
7.96346 0.870758
10.1891 1.12074
11.0471 1.19842
13.1088 1.36252
16.3759 1.59591
16.5678 1.42443
17.2971 1.09024];

alpha_von = von_doenhoff(:, 1);
cl_von2 = von_doenhoff(:, 2);

%% Fit: 'untitled fit 1'.
[xData, yData] = prepareCurveData( alpha_deg, cl );

% Set up fittype and options.
ft = fittype( 'poly1' );
excludedPoints = xData > 10;
opts = fitoptions( 'Method', 'LinearLeastSquares' );
opts.Lower = [-Inf 0];
opts.Normalize = 'On';
opts.Upper = [Inf 0];
opts.Exclude = excludedPoints;

% Fit model to data.
[fitresult, gof] = fit( xData, yData, ft, opts );

% Plot fit with data.
figure( 'Name', 'untitled fit 1' );
hold on
h = plot(fitresult, xData, yData, excludedPoints);
plot(alpha_deg(1:7), thin, 'LineWidth', 2, 'Color', 'Blue')
plot(alpha_von, cl_von2, 'LineWidth', 1, 'Color', 'Magenta')
legend('Calculated Lift Coefficient', 'Excluded Lift Coefficient', 'y = 0.5726 *
x', 'Thin Airfoil Theory', 'Abbott and von Doenhoff', 'Location', 'SouthEast');
title({'Lift Coefficient vs Angle of Attack', 'with Thin Airfoil Theory, von
Doenhoff, and Curve Fitting'})
xlabel( 'Angle of Attack');
ylabel( 'Lift Coefficient');
grid on
hold off
```

(1) The full MATLAB script that was used to analyze the data recorded by ScanTel. Output plots are found in Appendix B.

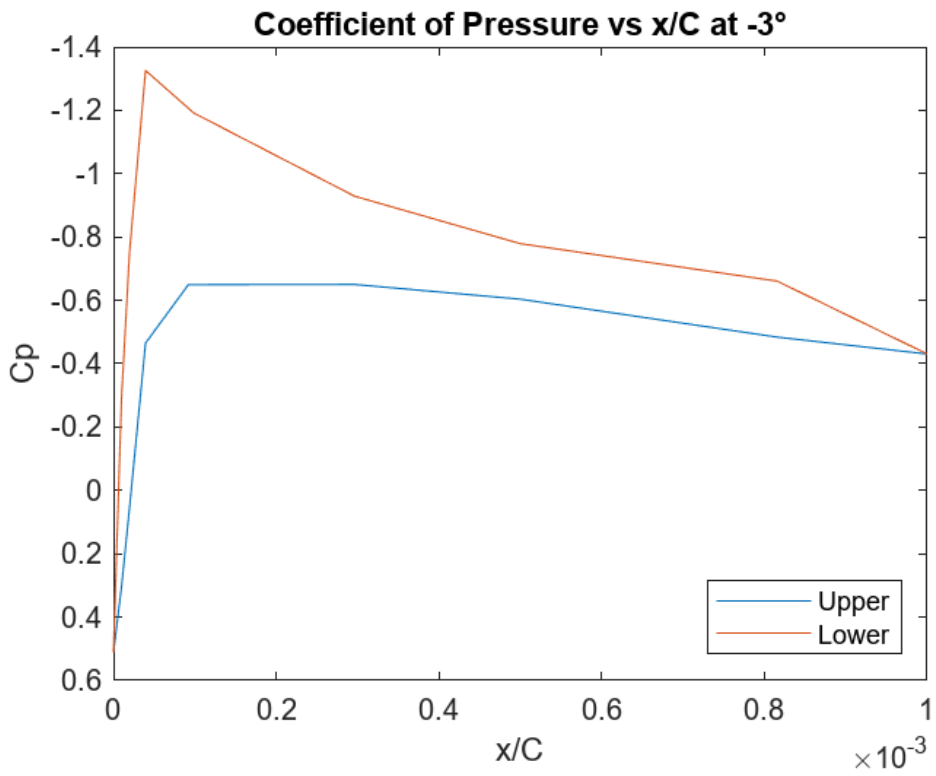
# Appendix B - MATLAB Figures



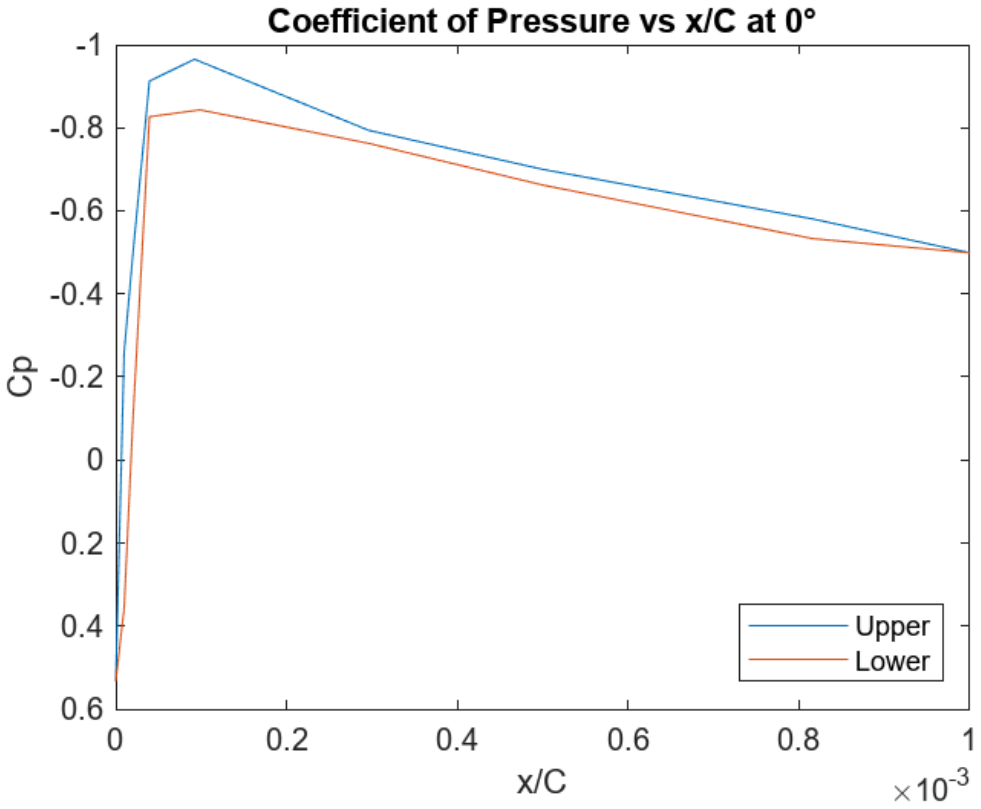

(1) Coefficient of pressure vs chord position for -10 degrees angle of attack



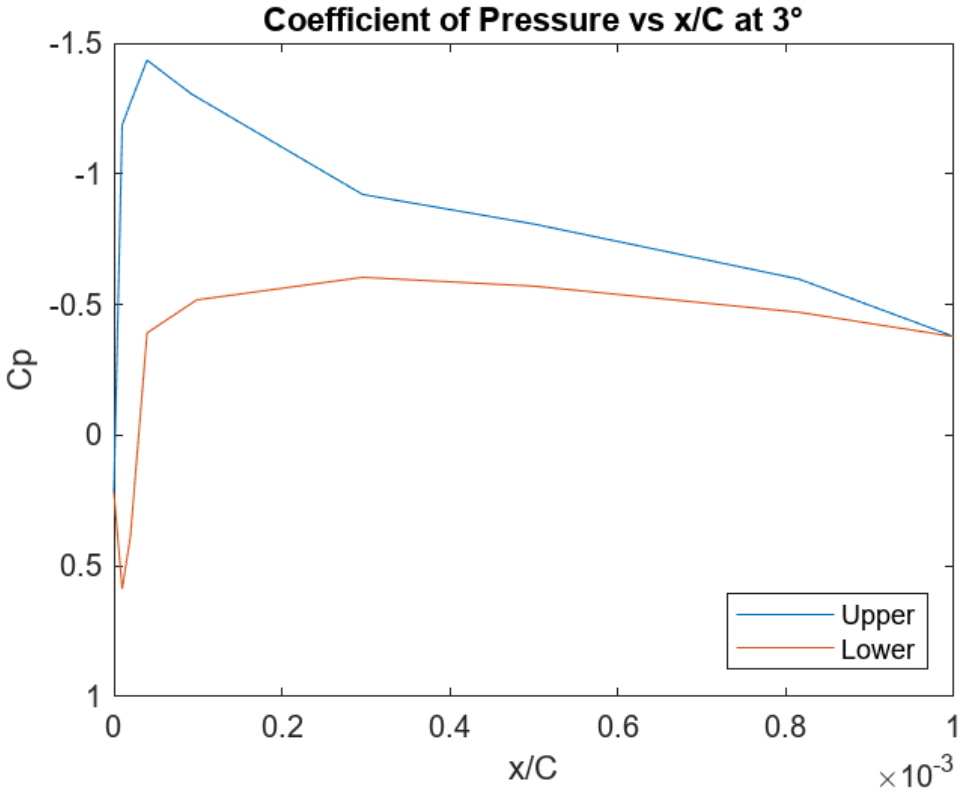

(2) Coefficient of pressure vs chord position for -6 degrees angle of attack



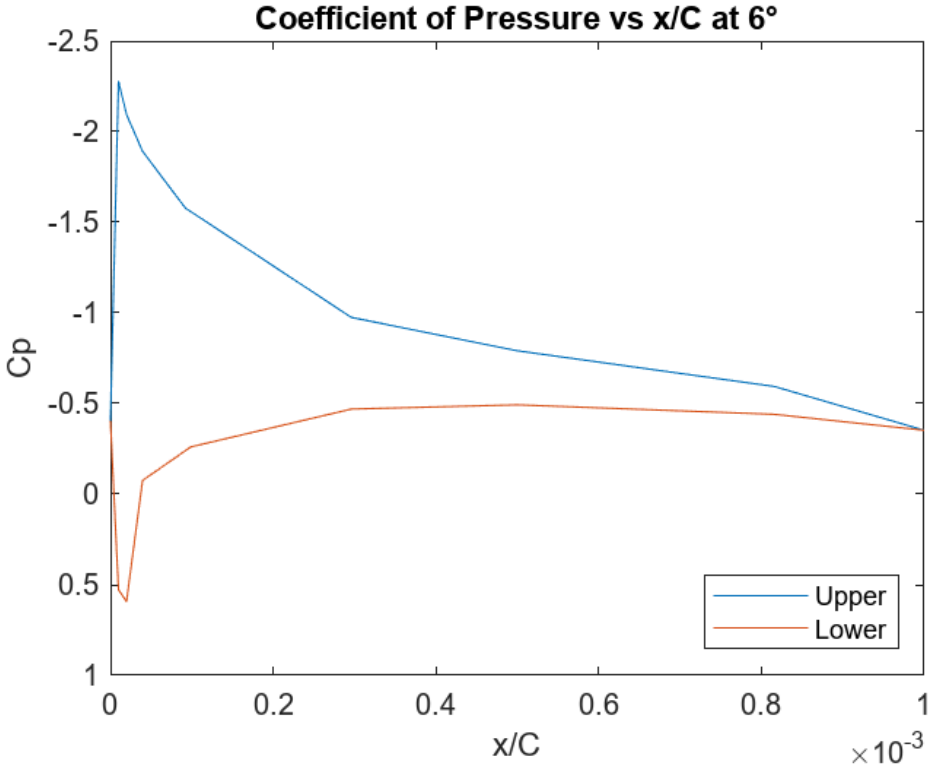

(3)Coefficient of pressure vs chord position for -3 degrees angle of attack



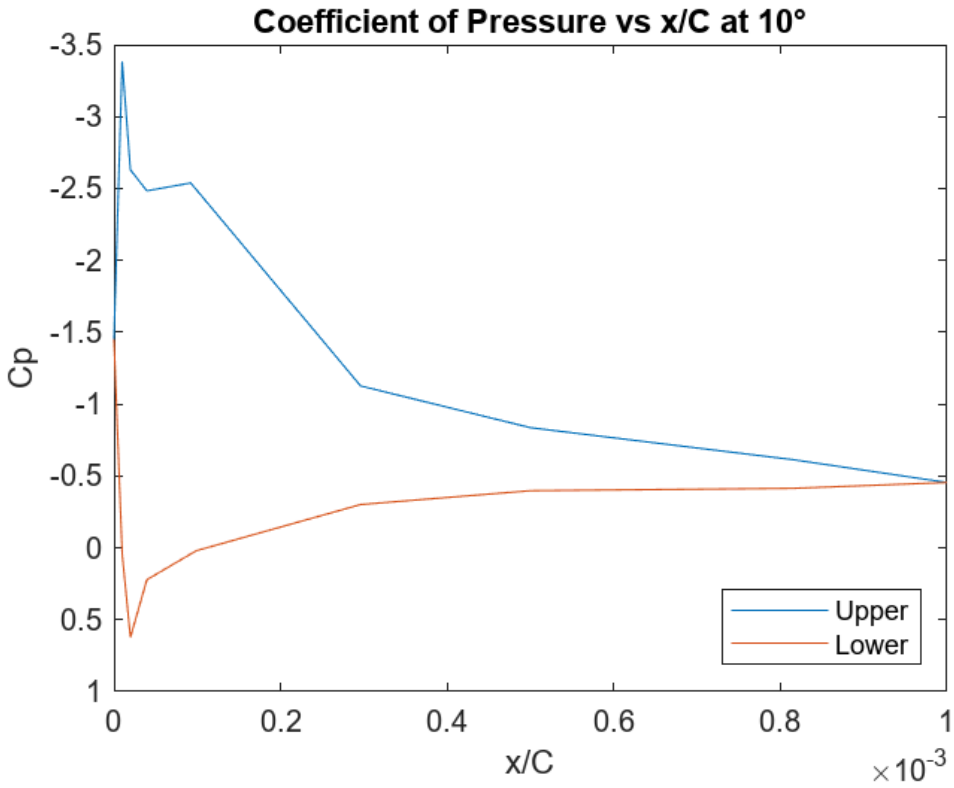

(4) Coefficient of pressure vs chord position for 0 degrees angle of attack



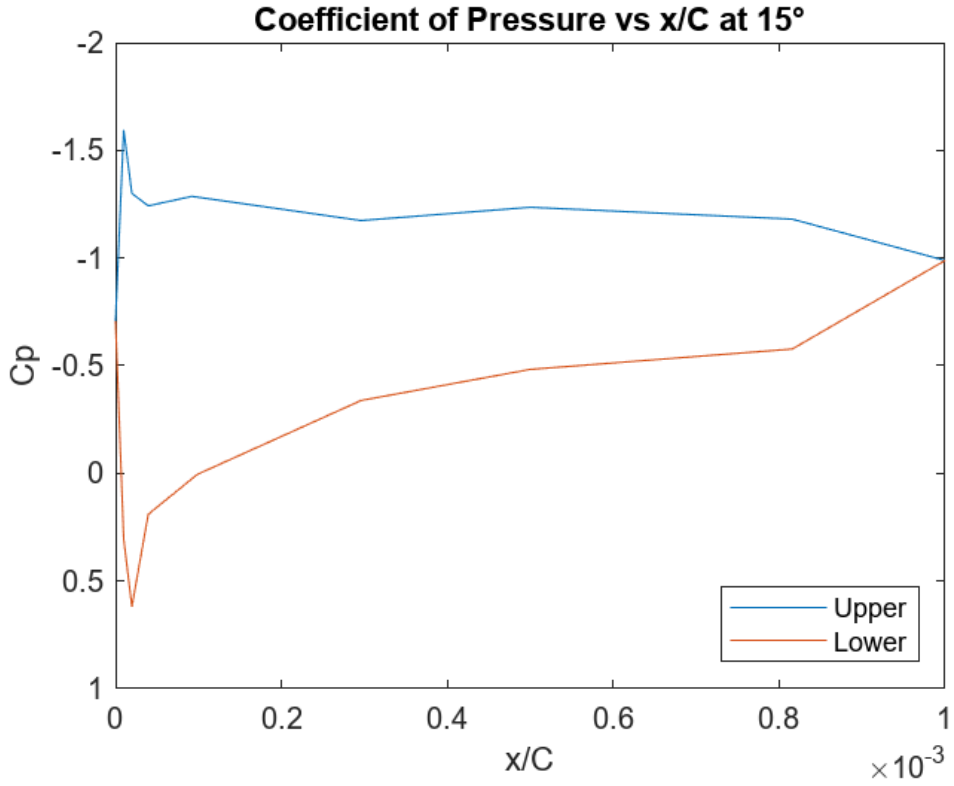

(5) Coefficient of pressure vs chord position for 3 degrees angle of attack



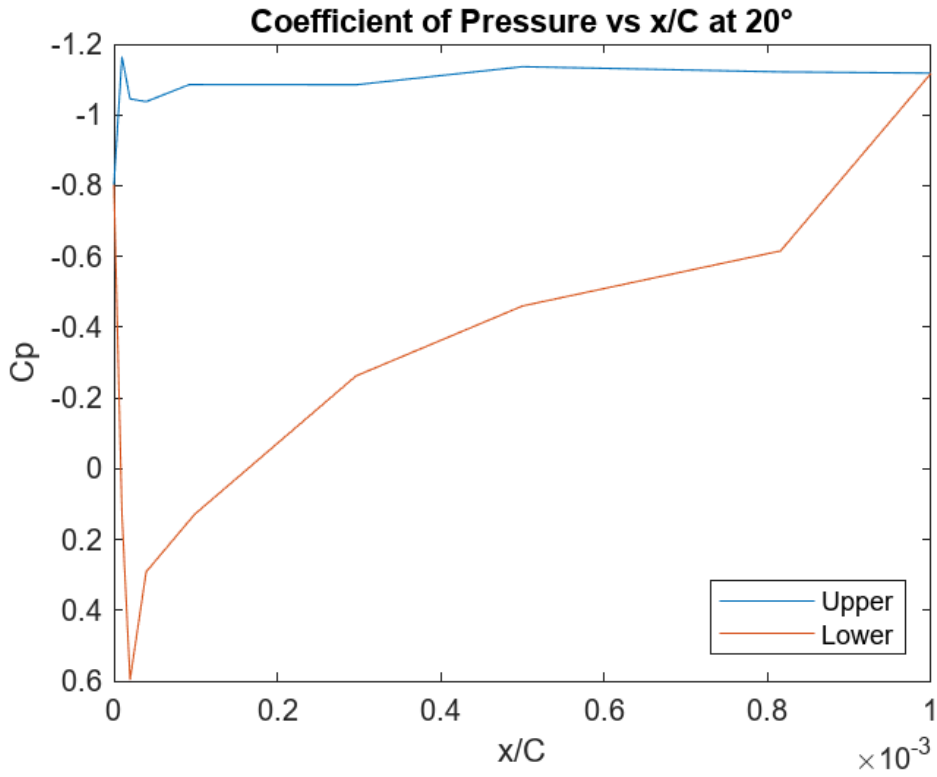

(6) Coefficient of pressure vs chord position for 6 degrees angle of attack



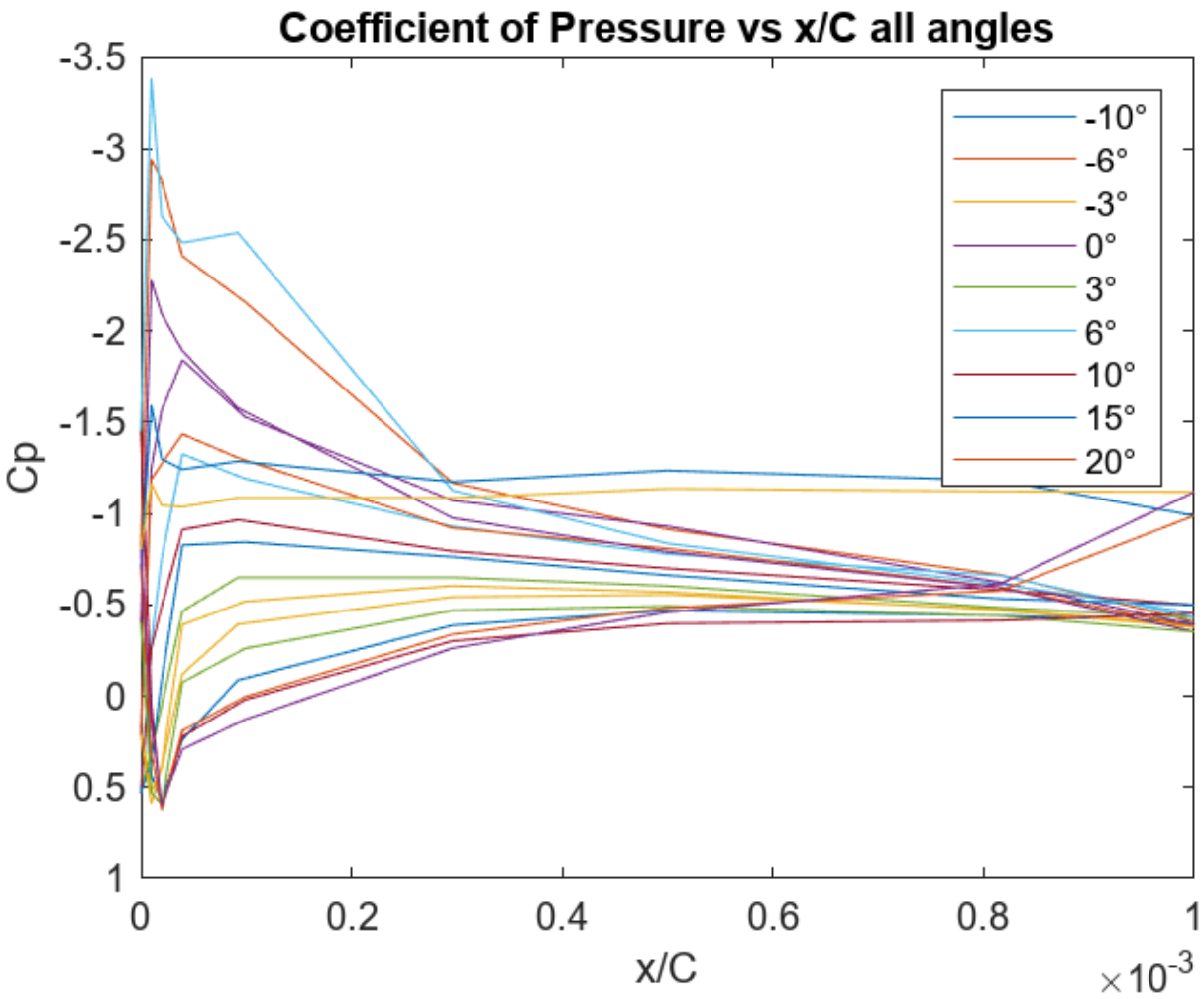

(7) Coefficient of pressure vs chord position for 10 degrees angle of attack



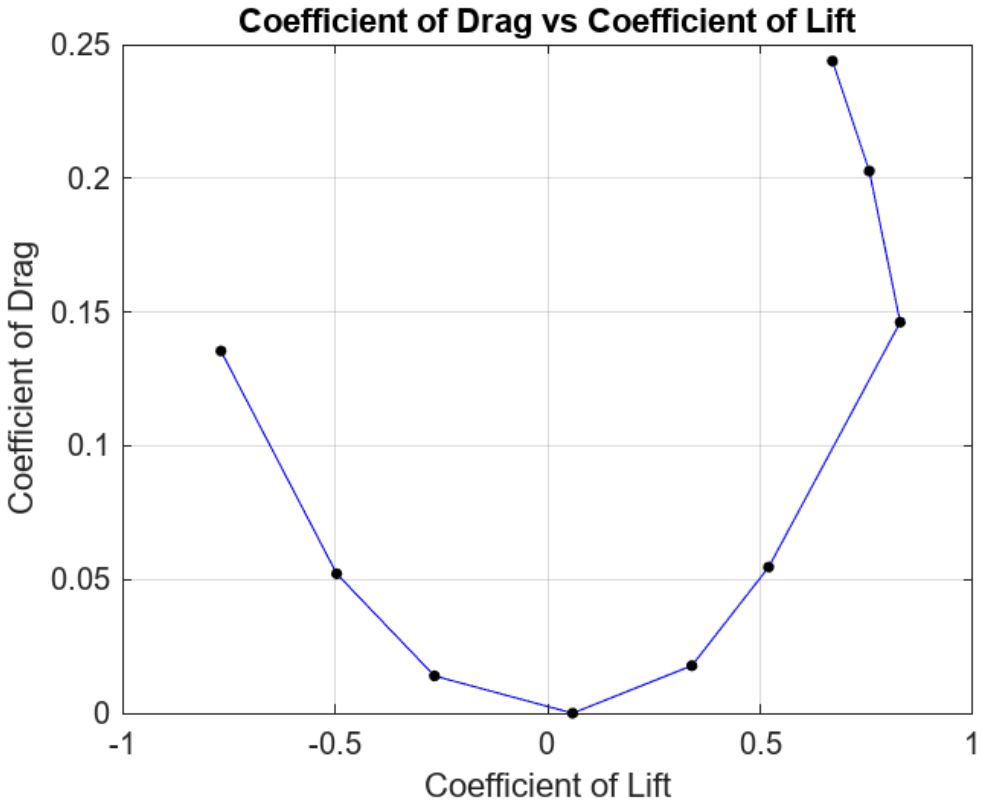

(8) Coefficient of pressure vs chord position for 15 degrees angle of attack



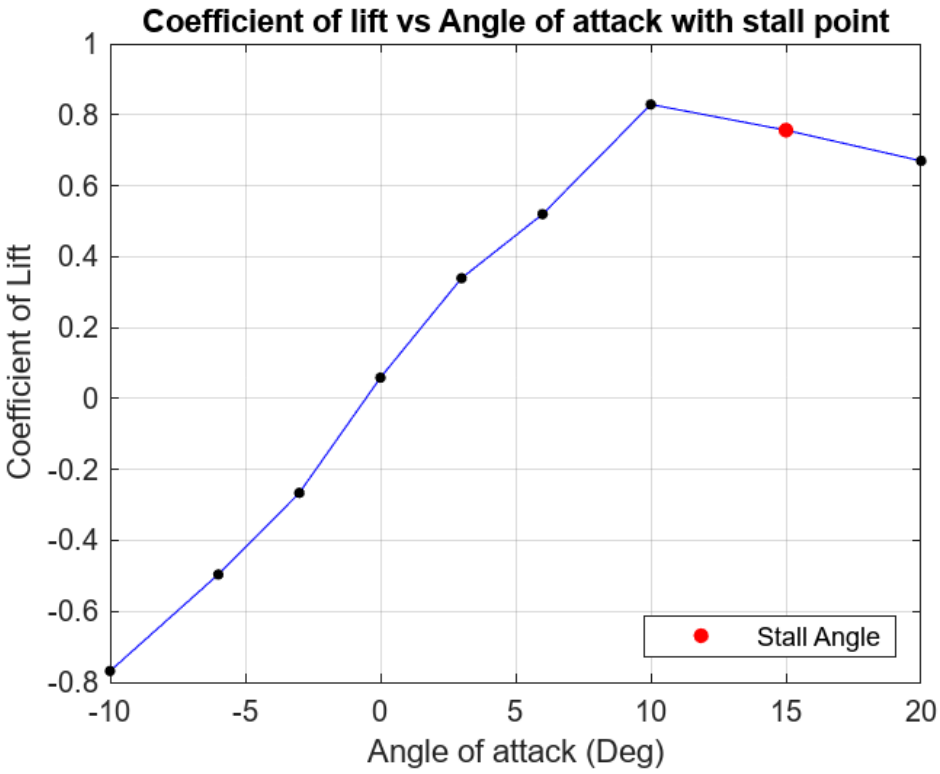

(9) Coefficient of pressure vs chord position for 20 degrees angle of attack



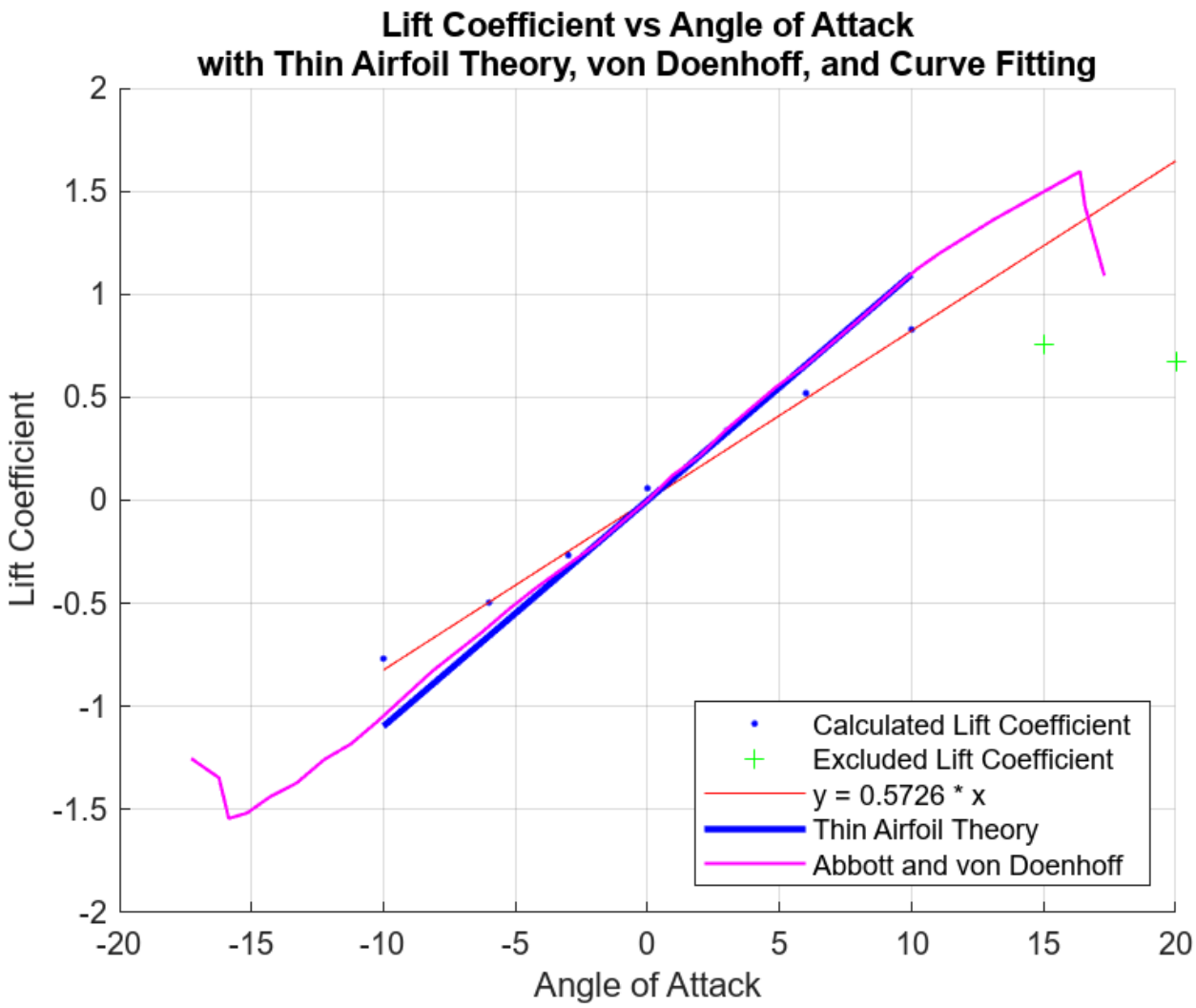

(10) Coefficient of pressure vs chord position for all angles of attack




(11) Coefficient of Drag vs Coefficient of Lift

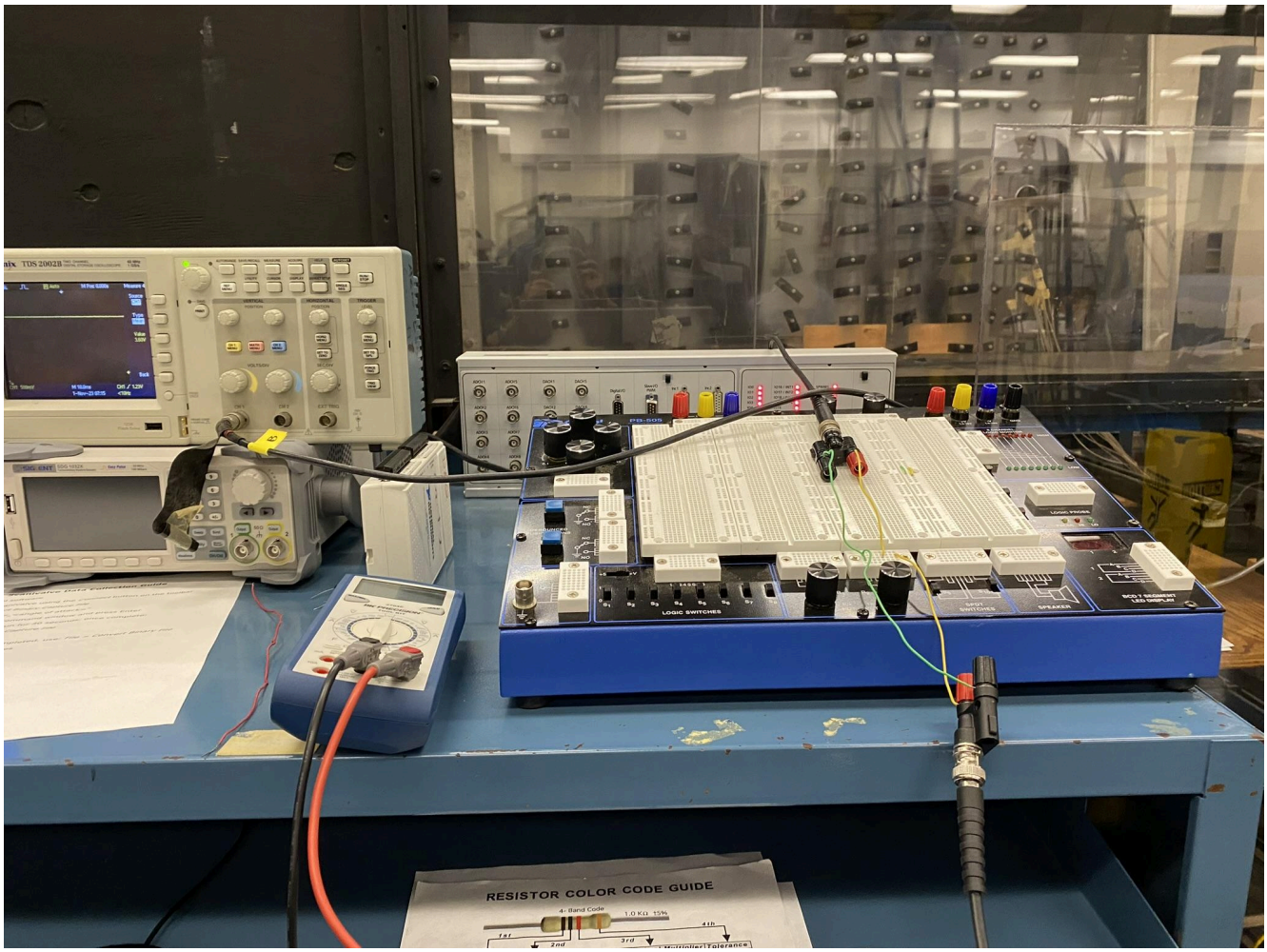



(12) Coefficient of Lift vs Angle of Attack with Stall Point

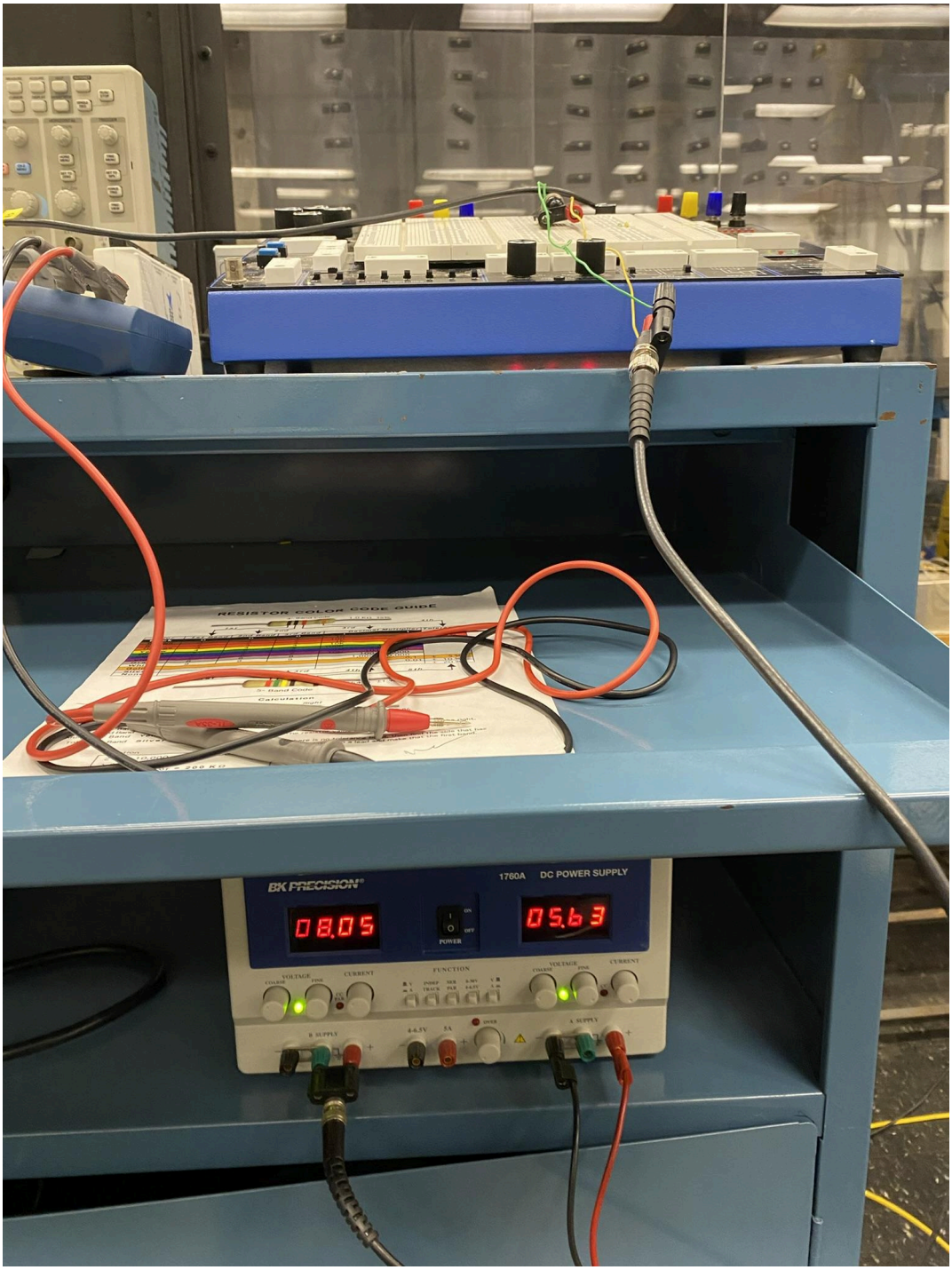



(13) Coefficient of Lift vs Angle of Attack with Curve Fitting, Thin Airfoil Theory, and Abbott and von Doenhoff Experimental Data

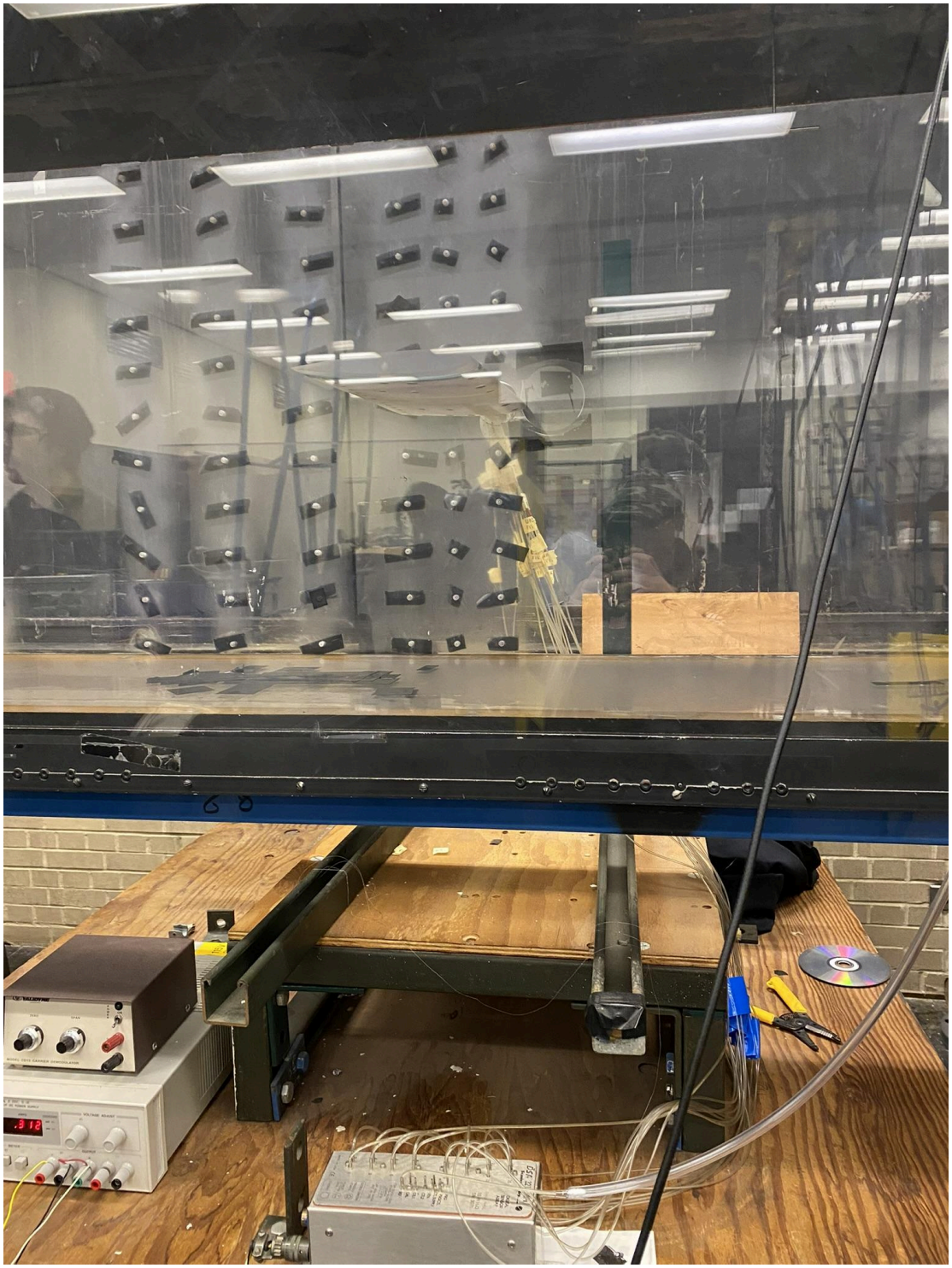
# Appendix C - Experimental Setup




(1) Connection between the computer and Scanivalve DSA 3217 for data capture by ScanTel.

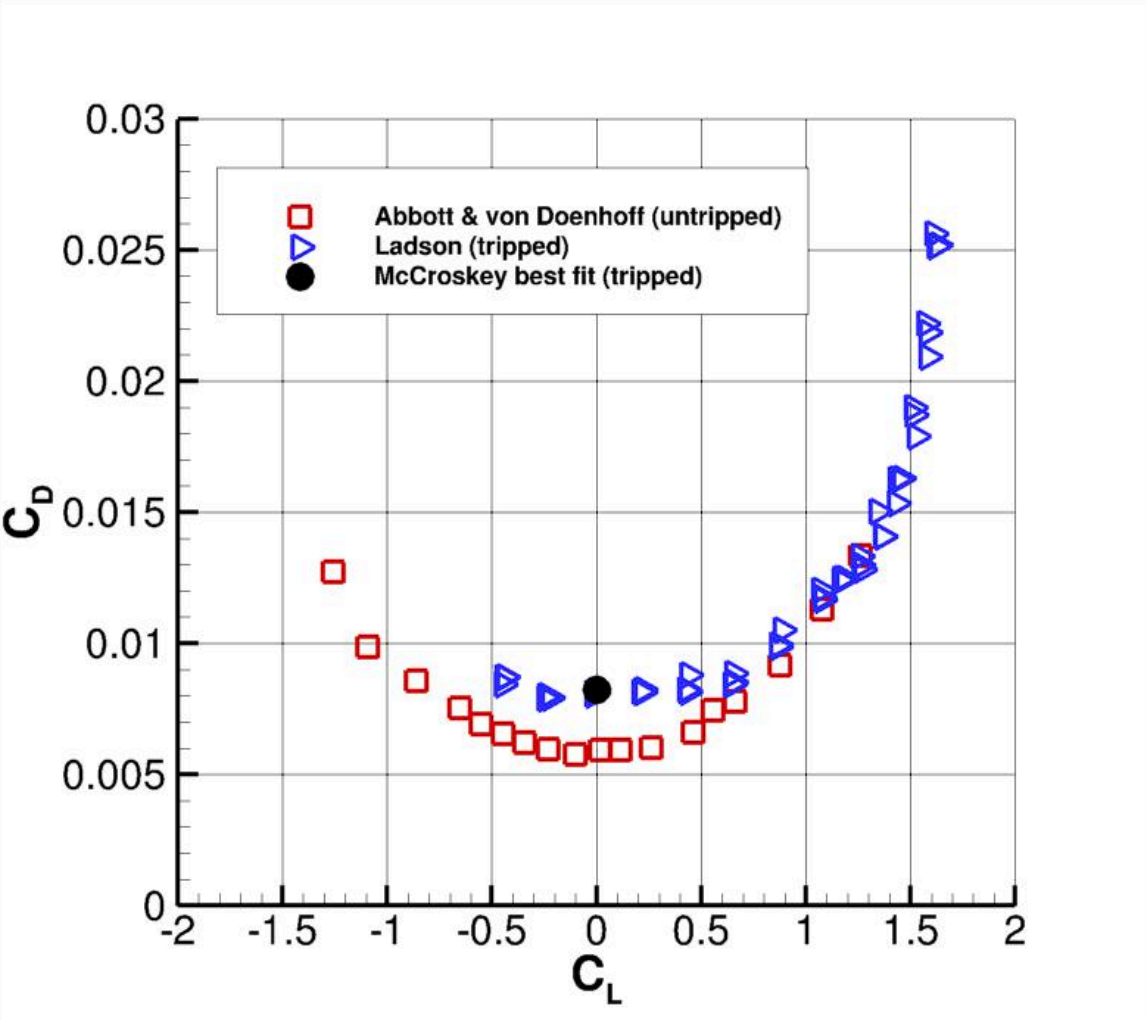



(2) Power supply setup and voltages for the wind tunnel.

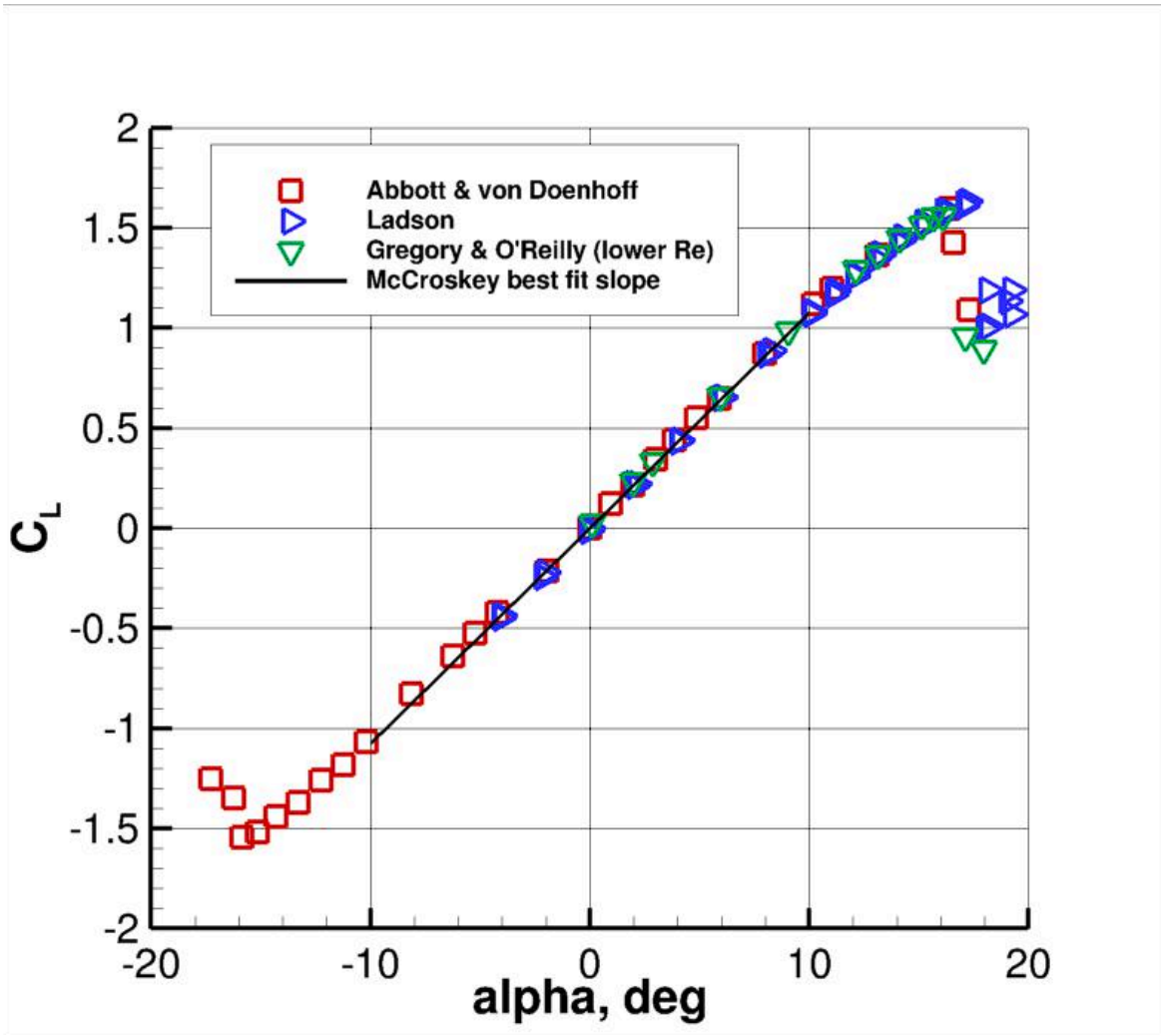


(3) Wind tunnel test section with NACA 0012 airfoil. Pressure taps can be seen running from the foil down the back side of the test section to the Scanivalve DSA 3217 at the bottom. Angle adjustment is controlled from the back side of the test section by manually rotating the airfoil.

# Appendix D - Published Results




(1) Coefficient of drag vs coefficient of lift based on Abbott and von Doenhoff's published results (3)




(2) Coefficient of lift vs angle of attack from Abbott and von Doenhoff published results (3)